

Preliminary Report For Project 7

Sentiment Analysis

Group 19

Nolan Capomaggio and Mathis Morra-Fischer

May 31, 2026

Abstract

Sentiment analysis is a key task in Natural Language Processing (NLP), aiming to automatically determine the emotional polarity of a given text. In this work, the goal is to address binary sentiment classification of Amazon product reviews. We distinguish positive (4–5 stars) from negative (1–2 stars) review. We use the Amazon Reviews dataset (Kaggle, Adam Bittlingmayer), which includes approximately 4 million labeled reviews. We propose to compare three approaches of increasing complexity: Logistic Regression with TF-IDF features, a LSTM neural network, and a fine-tuned RoBERTa model. Models are evaluated using accuracy and macro F1-score.

1 Introduction

For large companies as Amazon, being able to classify good and bad reviews is mandatory in order to be able to satisfy the customer. Knowing this, our goal is to develop a trustworthy solution to tackle this issue. Our solution is known as sentiment analysis.

In this project, we frame sentiment analysis as a **binary text classification** problem. Given the text of an Amazon product review, the goal is to predict a label $y \in \{0, 1\}$, where 0 denotes a negative review (1–2 stars) and 1 denotes a positive review (4–5 stars). Reviews rated 3 stars (neutral) are excluded from the dataset by construction, so no neutral class is considered.

We use the Amazon Reviews dataset published by Bittlingmayer on Kaggle, which provides 3.6 million training examples and 400,000 test examples in fastText format. We propose to compare three methods of increasing complexity: a Logistic Regression classifier with TF-IDF features, a Long Short-Term Memory (LSTM) recurrent neural network, and a fine-tuned RoBERTa transformer model.

2 Dataset

We use the Amazon Reviews dataset from Kaggle (<https://www.kaggle.com/datasets/bittlingmayer/amazonreviews>). Each line is formatted as a fast-Text label followed by the review text: `__label__1` corresponds to a negative review (1–2 stars) and `__label__2` corresponds to a positive review (4–5 stars). Three-star reviews are excluded by construction, so the task is strictly binary. It contains approximately 4 million labeled reviews.

2.1 Data Balance & Splits

The two classes are distributed equally: exactly 50% negative and 50% positive in both the training and test splits. The dataset is therefore **perfectly balanced**, and no oversampling or undersampling is required.

The dataset contains approximately 4 millions entries. It is already splitted into a training part and a test part with a ratio of 90/10 (which is enough because the testing part is already consituated of 400 000 entries). This is considered a standard for this kind of dataset.

2.2 Preprocessing

For all methods implemented we have to do a based preprocess. The `__label__X` prefix is stripped. The text itself requires cleaning because it is written by humans which are not reliable and use special characters. We will remove HTML tags, special characters and unnecessary punctuation. We will also convert text to lowercase to prevent taking uppercased words as different words.

For each models that we are going to use, we have to do a different preprocessing in order to make them more efficient.

- For Logistic Regression we will use TF-IDF (Term Frequency-Inverse Document Frequency). This turns text into a numerical matrix. It highlights words that are important to s pecific reviews.
- For LSTM, the text will be converted into sequences of integers. We will use padding to ensure all inputs have the same length. We will also use pre-trained word embeddings to represent semantic meaning.
- The RoBERTa is the one with the most advanced preprocessing. We have to use a byte-level Byte-Pair Encoding tokenizer. This handles rare words by breaking them into smaller sub-units. Then We will add the `<s>` (start of string) and `</s>` (separator) tokens. The model uses the representation of the first token for classification. And finally we have to create masks to tell the model which parts of the input are real text and which are only padding.

2.3 Evaluation Metrics

We want to collect the following metrics during tests:

- **Accuracy:** the proportion of correctly classified examples. This is a valid primary metric here because the dataset is perfectly balanced.
- **Macro F1-score:** the unweighted mean of per-class F1 scores. It provides a more robust basis for comparing methods, particularly as a cross-benchmark reference.
- **Precision and Recall** per class: to identify asymmetric error patterns (e.g., a model biased toward predicting positive).
- **Confusion matrix:** for qualitative analysis of error types.

We will compare our results to the top scores on the website. Based on the Kaggle leaderboard and existing literature for this dataset, the state-of-the-art accuracy is approximately 92–94%. Our goal is to approach this upper bound using the RoBERTa model.

3 Solution Description

3.1 Overview

For our project we will be using three different methods: *Logistic Regression* represent documents as sparse bag-of-words or TF-IDF vectors and train linear or kernel-based classifiers [?]. *LSTM* model is a sequential structure of text and capture order-dependent patterns; Long Short-Term Memory (LSTM) networks are the most used in this type of processing. *RoBERTa* such as BERT and its successors achieve state-of-the-art results combining large-scale unsupervised pre-training with task-specific fine-tuning.

The choice of method focuses on quantifying the performance gain at each level of complexity and to evaluate whether the additional computational cost of more advanced models is justified on this particular dataset.

3.2 Logistic Regression with TF-IDF

Logistic Regression trained on TF-IDF features serves as our baseline. The method is fast, fully interpretable, and has been shown to yield competitive accuracy on large-scale review datasets.

Feature extraction TF-IDF (Term Frequency – Inverse Document Frequency) assigns a weight to each word in a document that reflects how informative that word is relative to the entire corpus. We can construct a TF-IDF matrix using unigrams and bigrams, retaining the top 100,000 features ranked by term frequency. We fit a Logistic Regression classifier with ℓ_2 regularization using the `scikit-learn` implementation.

Motivation This method acts as a strong reference point: any more complex model must surpass it to justify its additional cost in training time and computational resources, it is a fast method (not time consuming) but not so efficient.

3.3 Long Short-Term Memory Network

LSTM networks are a class of recurrent neural networks designed to capture long-range dependencies in sequential data. Each LSTM cell maintains a memory state that is updated at every time step through learned gating mechanisms (input, forget, and output gates), which mitigate the vanishing gradient problem of vanilla RNNs.

Training. The model is trained with the Adam optimizer and binary cross-entropy loss for up to 5 epochs with early stopping based on validation F1-score for allowing training in a resonable time.

Motivation. Compared to the TF-IDF baseline, LSTM captures word order and local context. It represents an intermediate complexity level and does not require a GPU for inference, making it a practical middle choice.

3.4 RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) is a transformer encoder pre-trained on a large English text corpus. It improves over the original BERT model by removing the Next Sentence Prediction objective, and training with larger batches and more data. These modifications consistently improve downstream classification performance.

Architecture. We fine-tune the `roberta-base` checkpoint from the Hugging Face Transformers library. A linear classification head is added on top of the representation of the `<s>` (start-of-sequence) token, which aggregates contextual information from the entire input.

Fine-tuning. Fine-tuning is performed for up to 3 epochs with a learning rate of 2×10^{-5} , linear warmup over the first 10% of steps, and weight decay of 0.01. The batch size is set to 32.

Motivation. RoBERTa represents the state-of-the-art family of methods for text classification. Its inclusion allows us to measure the benefit of large-scale pre-trained contextual representations over task-specific architectures trained from scratch.

4 Midterm Solution

4.1 Implemented Progress

At this stage of the project, we have implemented and evaluated one complete method: **Logistic Regression with TF-IDF features**. This model is used as the baseline because it is fast to train, easy to reproduce, and provides a strong reference before implementing more expensive neural models.

The current implementation includes the following parts:

- a loader for the fastText dataset format, converting `__label__1` to class 0 and `__label__2` to class 1;
- a stratified train/validation split that keeps the 50/50 class balance;
- basic preprocessing, including HTML tag removal, lowercasing, and whitespace normalization;
- a `scikit-learn` pipeline with TF-IDF vectorization followed by Logistic Regression;
- evaluation code for accuracy, macro F1-score, precision, recall, classification reports, and confusion matrices;
- saving the trained model as a `.pkl` file for later reuse.

The Logistic Regression model uses unigram and bigram TF-IDF features, a maximum vocabulary size of 100,000 features, sublinear term-frequency scaling, and ℓ_2 regularization with $C = 1.0$.

4.2 Reproducibility

The experiment is reproducible because the data split uses a fixed random seed. The default seed is **42**, and it can be changed through the command line argument `-seed`. The full baseline experiment was run with:

```
python3 sentiment/train.py \  
--model logistic \  
--train sentiment/data/train.ft.txt \  
--test sentiment/data/test.ft.txt \  
--output_dir sentiment/out/logistic_full
```

The full Kaggle training file contains 3,600,000 reviews. We split it into 2,880,000 training examples and 720,000 validation examples. The official test file contains 400,000 reviews.

Table 1: Midterm results for Logistic Regression with TF-IDF.

Split	Examples	Accuracy	Macro F1	Precision	Recall
Validation	720,000	0.9413	0.9413	0.9413	0.9413
Test	400,000	0.9407	0.9407	0.9407	0.9407

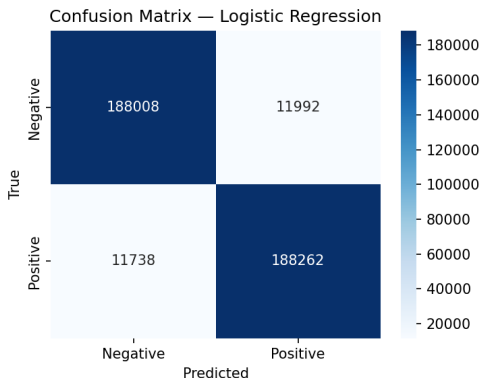


Figure 1: Confusion matrix for the Logistic Regression baseline on the full test set.

4.3 Intermediate Results

Table 1 presents the current results on the full dataset. The validation and test results are very close, which suggests that the model generalizes consistently and that the split is representative.

The model obtains 94.07% accuracy and 94.07% macro F1-score on the full test set. Precision and recall are also balanced between the negative and positive classes, both being approximately 0.94. This shows that the model does not have a strong bias toward one class, which is expected because the dataset is perfectly balanced.

4.4 Challenges, Findings, and Corrections

The first challenge was practical dataset handling. The Kaggle files are large and must be extracted before training. To make this easier to diagnose, the loader now raises a clearer error message when `train.ft.txt` or `test.ft.txt` is missing.

The second challenge was environment compatibility. The local virtual environment used Python 3.9, so Python 3.10-only annotations such as `str | None` were replaced by compatible alternatives. We also configured Matplotlib to use a non-interactive backend so that confusion matrices can be saved reliably from the terminal.

The main finding so far is that the baseline is stronger than expected. In the initial plan, Logistic Regression was presented mainly as a fast but less powerful baseline. However, it already reaches around 94% test accuracy on the full dataset. This means that LSTM and RoBERTa must be evaluated not only by accuracy, but also by training time, computational cost, and practical benefit over this strong baseline.

4.5 Plan for the Final Report

For the final report, we plan to complete the comparison between the three selected methods: Logistic Regression, LSTM, and RoBERTa. The final experiments should include:

- accuracy, macro F1-score, precision, recall, and confusion matrices for all implemented models;
- comparison of training time and hardware requirements;
- analysis of whether the LSTM and RoBERTa models improve enough over Logistic Regression to justify their additional complexity;
- qualitative error analysis on selected misclassified reviews;
- final discussion of the best trade-off between performance, interpretability, and computational cost.

5 Conclusion

We have described the dataset, evaluation protocol, and three methods for binary sentiment classification of Amazon product reviews. The Amazon Reviews dataset is large, perfectly balanced, and requires only minimal preprocessing; no class rebalancing technique is needed. Our three methods – Logistic Regression with TF-IDF, LSTM, and RoBERTa – span the full range from a lightweight and interpretable baseline to a state-of-the-art transformer model, enabling a controlled comparison of performance versus computational cost. In the next stage of the project, we will implement all three pipelines, compare their accuracy and macro F1-score on the validation set, and investigate the impact of key design choices such as vocabulary size, sequence length, and fine-tuning strategy.

References

- [1] NeurIPS (2024) Call for Papers
<https://neurips.cc/Conferences/2024/CallForPapers>
- [2] Adam Bittlingmayer (2020) Amazon Reviews for Sentiment Analysis
<https://www.kaggle.com/datasets/bittlingmayer/amazonreviews>